

Formal Verification Report: AARC iBTCY/BTCY Protocol

- Date: February 16th, 2026
 - Audit Repo: <https://github.com/Cyfrin/audit-2026-02-aarc-btcy>
 - Author: Specialist AI created by [@DevDacian](#) ([@cyfrin](#) private audit)
 - Certora Prover version: 8.8.0
-

Table of Contents

1. [About iBTCY/BTCY](#)
 2. [Formal Verification Methodology](#)
 - [Types of Properties](#)
 - [Project Structure](#)
 - [Assumptions](#)
 3. [Verification Properties](#)
 - [iBTCY \(iBTCY Token\)](#)
 - [BTCY \(ERC-4626 Vault\)](#)
 - [iBTCYHub \(Subscription/Redemption Hub\)](#)
 - [Pricer \(Price Oracle\)](#)
 - [DepositWithdraw \(Treasury\)](#)
 4. [Setup and Execution](#)
 - [Common Setup \(Steps 1-4\)](#)
 - [Remote Execution](#)
 - [Local Execution](#)
 - [Running Verification](#)
-

About iBTCY/BTCY

The iBTCY/BTCY protocol is a tokenized Bitcoin yield product. The system consists of five core contracts:

- **iBTCY** — An ERC-20 token with allowlist enforcement, pause functionality, compliance-driven force transfers, and vault protection. Uses transient storage for force transfer state management.
- **BTCY** — An ERC-4626 vault wrapper around iBTCY with withdrawal fees, transfer deny list, minimum deposit/withdraw amounts, and donation attack protection via MIN_SHARES.
- **iBTCYHub** — A subscription and redemption hub managing the lifecycle of deposit and redemption requests, with fee bounds, pause controls, and price-based processing.
- **Pricer** — A price oracle contract that stores price entries with Chainlink validation, rate limiting, deviation bounds, and sequencer uptime checks.
- **DepositWithdraw** — A treasury deposit/withdraw contract with token and user whitelisting, pause controls, and role-based authorization.

All contracts use OpenZeppelin's upgradeable access control (`AccessControlEnumerableUpgradeable`) except `DepositWithdraw` which uses non-upgradeable `AccessControlEnumerable`. Storage follows the ERC-7201 namespaced pattern for upgradeable contracts.

Formal Verification Methodology

Certora Formal Verification (FV) provides mathematical proofs of smart contract correctness by verifying code against a formal specification. Unlike testing and fuzzing which examine specific execution paths, Certora FV examines all possible states and execution paths.

The process involves crafting properties in CVL (Certora Verification Language) and submitting them alongside compiled Solidity smart contracts to a remote prover. The prover transforms the contract bytecode and rules into a mathematical model and determines the validity of rules.

Types of Properties

Invariants — System-wide properties that **MUST** always hold true. These are parametric — automatically verified against every external function in the contract. Once proven, invariants serve as trusted assumptions via `requireInvariant`.

Parametric Rules — Rules verified against every non-view external function using method `f` with `callDataarg` `args`. Used for properties like "only mint can increase `totalSupply`" or "counter values never decrease."

Access Control Rules — Rules verifying that state-changing functions revert when the caller lacks the required role. Uses the `@withrevert` pattern: call the function, then assert `!lastReverted => hasRole(...)`.

Revert Condition Rules — Rules verifying that functions revert under specific invalid conditions (zero inputs, paused state, missing allowlist, etc.). Uses `@withrevert` followed by `assert !lastReverted`.

Integrity Rules — Rules verifying that successful function calls produce the correct state changes (e.g., transfer moves exact amounts, deposit records match inputs, preview functions match actual operations, round-trip conversions never inflate value).

Sanity (Satisfy) Rules — Lightweight reachability checks ensuring functions are not vacuously verified. Uses `satisfy true` to confirm at least one non-reverting execution path exists.

Project Structure

```
certora/
├── conf/                                # Prover configuration files
│   ├── IBTCY.conf
│   ├── BTCY.conf
│   ├── BTCY_EIP4626.conf              # Separate conf for DISPATCHER(true) rules
│   ├── IBTCYHub.conf
│   ├── Pricer.conf
│   └── DepositWithdraw.conf
├── harnesses/                          # Verification harnesses
│   ├── IBTCYHarness.sol               # Exposes transient _isForceTransferring via tload
│   ├── BTCYHarness.sol                # Exposes selfAddress() for address(this) access
│   ├── IBTCYHubHarness.sol            # Exposes struct fields from depositIdToDepositor/redemptionIdToRedeemer
│   ├── PricerHarness.sol              # Exposes individual price/timestamp from prices() tuple
│   └── DepositWithdrawHarness.sol     # Minimal harness (constructor passthrough)
├── specs/                             # CVL specifications
│   ├── IBTCY.spec                    # 18 properties: token conservation, access control, force transfer, allowlist
│   ├── BTCY.spec                     # 46 properties: fee bounds, deny list, allowlist, MIN_SHARES, pause, admin safety
│   ├── BTCY_EIP4626.spec             # 19 properties: EIP-4626 compliance, round-trip, integrity, solvency, fee
│   ├── IBTCYHub.spec                 # 22 properties: counter monotonicity, fee bounds, request lifecycle, pause
│   ├── Pricer.spec                   # 16 properties: price validation, monotonicity, rate limiting, admin setters
│   └── DepositWithdraw.spec           # 12 properties: whitelist enforcement, authorization, treasury safety
```

Assumptions

Safe Assumptions

These reflect real-world constraints that do not reduce security coverage.

Environment Constraints (all specs):

- `msg.sender` is never the zero address (Ethereum protocol guarantee)
- Block timestamps are within realistic bounds

Conservation Constraints (IBTCY, BTCY):

- No individual balance exceeds `totalSupply` — this is a mathematical invariant of ERC-20 implementations but cannot be expressed as a standalone invariant in Certora without a full conservation proof, so it appears as a `require` in rules that need it (e.g., transfer integrity, state-transition integrity, solvency)
- Asset token transfer pairs have bounded sum (`bal(a) + bal(b) <= max_uint256`) — prevents OZ's unchecked `_balances[to] += value` from overflowing in the prover's arbitrary starting states

mulDiv Tractability (BTCY_EIP4626):

- Inputs to OZ Math.`mulDiv` are bounded to `max_uint112` — when `a * b` overflows `uint256`, `mulDiv` uses bitwise operations the prover cannot reason about precisely. `max_uint112` ensures `a * b < 2^256` while far exceeding any realistic token supply ($\approx 5.2e33$). This applies to monotonicity, round-trip, and fee integrity rules.

Address Separation (IBTCYHub, BTCY):

- The `asset()` address is linked to the concrete IBTCY harness via `struct_link`, ensuring the prover resolves external calls to the correct contract
- from `!=` compliance in force transfer rules to test the non-trivial case where balances actually change

Transient Storage

Certora rules execute mid-transaction, so EIP-1153 transient storage is NOT zeroed at rule start. The `_isForceTransferring` private transient variable in IBTCY is exposed via an assembly `tload(0)` getter in the harness, and rules that depend on its initial value explicitly require it to be `false` (matching the EIP-1153 guarantee that transient storage starts at zero at transaction boundaries).

External Call Summaries

- **Chainlink calls** (`latestRoundData`, `getRoundData`): Summarized as `NONDET` in `Pricer.spec` — we verify `Pricer` logic, not Chainlink correctness
- **ERC-20 calls** (`safeTransferFrom`, `safeTransfer`, `balanceOf`): Summarized as `NONDET` in `DepositWithdraw.spec`
- **Cross-contract calls** (IBTCYHub, BTCY): Use `DISPATCH` to linked harnesses with `NONDET` fallback
- **DISPATCHER(true)** (BTCY_EIP4626): Forces optimistic dispatch for `_.balanceOf`, `_.transfer`, `_.transferFrom` — required because ERC-7201 namespaced storage causes each `IERC20(asset()).balanceOf()` call to be havoced independently, breaking consistency between preview functions and actual operations

DISPATCHER(true) Limitation

Solvency after withdraw/redeem (B-62/B-63) was dropped because `DISPATCHER(true)` picks callees independently per call site. Withdraw and redeem have multiple internal `IERC20(asset()).balanceOf(address(this))` calls (via `maxRedeem` → `previewRedeem` and `redeem` →

previewRedeem). The DISPATCHER can resolve one to IBTCYHarness and another to BTCYHarness, producing inconsistent totalAssets values within a single execution. Combined with the property not being inductive in arbitrary starting states, these rules are infeasible with current tooling. Solvency after deposit/mint (B-60/B-61) works because deposits always add assets, preserving solvency regardless of DISPATCHER choice.

Verification Properties

133 properties across 5 contracts. All properties verified successfully.

- **Invariants:** 5
- **Parametric rules:** 9
- **Access control rules:** 21
- **Revert condition rules:** 63
- **Integrity rules:** 27
- **Sanity rules:** 8

IBTCY (iBTCY Token)

18 properties covering token supply conservation, access control, transfer integrity, vault protection, force transfer safety, allowlist enforcement, and admin safety.

ID	Name	Type	Description
I-2	onlyMintIncreasesTotalSupply	Parametric	No function other than mint can increase totalSupply
I-3	onlyBurnDecreasesTotalSupply	Parametric	Only burn functions can decrease totalSupply
I-4	onlyMinterCanMint	Access Control	mint reverts if caller lacks MINTER_ROLE
I-5	onlyBurnerCanBurnFrom	Access Control	burn(address, uint256) reverts if caller lacks BURNER_ROLE
I-6	transferIntegrity	Integrity	Transfer: sender decreases, receiver increases by exact amount
I-7	btcyVaultTransferProtection	Revert Condition	Transfer to btcyVault reverts unless sender is the vault itself
I-8	forceTransferAuthAndDestination	Access Control + Integrity	forceTransfer requires COMPLIANCE_ROLE and sends tokens to complianceAddress
I-9	forceTransferRevertsOnInvalidInputs	Revert Condition	forceTransfer reverts on zero compliance address, zero from, or zero amount
I-10	cannotRenounceLastAdmin	Revert Condition	Cannot renounce the last DEFAULT_ADMIN_ROLE
I-11	cannotRevokeLastAdmin	Revert Condition	Cannot revoke the last DEFAULT_ADMIN_ROLE
I-12	mintSanity	Sanity	mint is reachable (at least one non-reverting path exists)
I-13	forceTransferSanity	Sanity	forceTransfer is reachable
I-14	burnRevertIfTargetIsBtcyVault	Revert Condition	burn reverts if target is btcyVault (vault shares must not be burned)
I-15	forceTransferRevertsIfTargetIsBtcyVault	Revert Condition	forceTransfer reverts if target is btcyVault
I-16	onlyAdminCanSetBTCYVault	Access Control	Only DEFAULT_ADMIN_ROLE can call setBTCYVault
I-17	onlyComplianceCanSetComplianceAddress	Access Control	Only COMPLIANCE_ROLE can call setComplianceAddress
I-18	transferRevertsIfSenderNotAllowlisted	Revert Condition	transfer reverts if sender is not allowlisted (requires transient storage handling)
I-19	setComplianceAddressRevertsOnZero	Revert Condition	setComplianceAddress reverts on zero address

BTCY (ERC-4626 Vault)

65 properties across 2 spec files covering EIP-4626 compliance, fee bounds, deny list enforcement, allowlist enforcement, MIN_SHARES protection, deposit/mint/withdraw/redeem integrity, round-trip rounding, solvency, zero-input validation, stealth donation protection, paused state behavior, max function conditions, self-transfer protection, and admin safety.

Rules that compare view functions to actual operations (preview == actual, monotonicity, round-trip, fee integrity, state-transition integrity, solvency, max upper bounds) require DISPATCHER(true) to prevent independent havoc on IERC20(asset()).balanceOf() calls caused by ERC-7201 namespaced storage. These are in a separate spec file (BTCY_EIP4626.spec).

ID	Name	Type	Description
B-1	withdrawalFeeBounded	Invariant	withdrawalFee is always bounded by MAX_FEE_BPS
B-2	minSharesAfterDeposit	Integrity	After deposit, totalSupply >= MIN_SHARES (donation attack protection)
B-3	transferRevertsIfSenderDenylisted	Revert Condition	Transfer reverts if sender is denylisted
B-4	transferRevertsIfReceiverDenylisted	Revert Condition	Transfer reverts if receiver is denylisted
B-5	transferToIBTCYReverts	Revert Condition	Transfer to iBTCY contract (asset()) always reverts
B-6	withdrawRevertsIfOwnerNotAllowedlisted	Revert Condition	withdraw reverts if owner is not allowlisted
B-7	depositIncreasesShares	Integrity	Deposit increases receiver's share balance by the returned amount; shares always nonzero
B-8	onlyAdminCanSetWithdrawalFee	Access Control	Only DEFAULT_ADMIN_ROLE can change withdrawalFee
B-9	onlyAdminCanSetFeeRecipient	Access Control	Only DEFAULT_ADMIN_ROLE can set feeRecipient
B-10	onlyComplianceCanUpdateDenyList	Access Control	Only COMPLIANCE_ROLE can update transfer deny list
B-11	cannotRenounceLastAdmin	Revert Condition	Cannot renounce the last DEFAULT_ADMIN_ROLE
B-12	cannotRevokeLastAdmin	Revert Condition	Cannot revoke the last DEFAULT_ADMIN_ROLE
B-13	depositSanity	Sanity	deposit is reachable
B-14	maxDepositZeroWhenPaused	Revert Condition	maxDeposit returns 0 when paused
B-15	maxMintZeroWhenPaused	Revert Condition	maxMint returns 0 when paused
B-16	maxWithdrawZeroWhenPaused	Revert Condition	maxWithdraw returns 0 when paused
B-17	maxRedeemZeroWhenPaused	Revert Condition	maxRedeem returns 0 when paused
B-18	transferToSelfReverts	Revert Condition	Transfer to address(this) always reverts
B-19	onlyAdminCanSetMinDepositAmount	Access Control	Only DEFAULT_ADMIN_ROLE can call setMinDepositAmount
B-20	depositRevertsIfBelowMinDeposit	Revert Condition	deposit reverts if assets below minDepositAmount
B-21	depositRevertsOnZeroAssets	Revert Condition	deposit(0, receiver) always reverts
B-22	mintRevertsOnZeroShares	Revert Condition	mint(0, receiver) always reverts
B-23	redeemRevertsOnZeroShares	Revert Condition	redeem(0, receiver, owner) always reverts
B-24	withdrawRevertsOnZeroAssets	Revert Condition	withdraw(0, receiver, owner) always reverts
B-25	mintCostsAssets	Integrity	mint always costs nonzero assets (stealth donation protection)
B-26	withdrawCostsShares	Integrity	withdraw always burns nonzero shares (stealth donation protection)
B-27	redeemProducesValue	Integrity	redeem always produces value — receiver or fee recipient gets paid
B-28	minSharesAfterMint	Integrity	After mint, totalSupply >= MIN_SHARES (donation attack protection)

B-30	previewDepositMatchesDeposit	Integrity	previewDeposit(assets) equals actual deposit return value
B-31	previewMintMatchesMint	Integrity	previewMint(shares) equals actual mint return value
B-32	previewRedeemMatchesRedeem	Integrity	previewRedeem(shares) equals actual redeem return value
B-33	previewWithdrawMatchesWithdraw	Integrity	previewWithdraw(assets) equals actual withdraw return value
B-34	convertToSharesMonotonicity	Integrity	convertToShares is monotonically non-decreasing
B-35	redeemRevertsIfOwnerNotAllowlisted	Revert Condition	redeem reverts if owner is not allowlisted
B-36	transferFromRevertsIfSenderDenylisted	Revert Condition	transferFrom reverts if sender is denylisted
B-37	transferFromRevertsIfReceiverDenylisted	Revert Condition	transferFrom reverts if receiver is denylisted
B-38	maxDepositZeroIfReceiverDenylisted	Revert Condition	maxDeposit returns 0 if receiver is denylisted
B-39	maxMintZeroIfReceiverDenylisted	Revert Condition	maxMint returns 0 if receiver is denylisted
B-40	maxWithdrawZeroIfOwnerDenylisted	Revert Condition	maxWithdraw returns 0 if owner is denylisted
B-41	maxRedeemZeroIfOwnerDenylisted	Revert Condition	maxRedeem returns 0 if owner is denylisted
B-42	maxWithdrawZeroIfOwnerNotAllowlisted	Revert Condition	maxWithdraw returns 0 if owner is not allowlisted
B-43	maxRedeemZeroIfOwnerNotAllowlisted	Revert Condition	maxRedeem returns 0 if owner is not allowlisted
B-44	previewRedeemNeverExceedsFreeConversion	Integrity	previewRedeem(shares) <= convertToAssets(shares) (withdrawal fee is non-negative)
B-45	setFeeRecipientRevertsOnZero	Revert Condition	setFeeRecipient reverts on zero address
B-46	setWithdrawalFeeRevertsIfNoRecipient	Revert Condition	Cannot set non-zero withdrawalFee without a fee recipient
B-47	withdrawRevertsIfBelowMinWithdraw	Revert Condition	withdraw reverts if assets below minWithdrawAmount (when no fee)
B-48	onlyAdminCanSetMinWithdrawAmount	Access Control	Only DEFAULT_ADMIN_ROLE can call setMinWithdrawAmount
B-49	depositRevertsWhenPaused	Revert Condition	deposit reverts when paused
B-50	mintRevertsWhenPaused	Revert Condition	mint reverts when paused
B-51	withdrawRevertsWhenPaused	Revert Condition	withdraw reverts when paused
B-52	redeemRevertsWhenPaused	Revert Condition	redeem reverts when paused
B-53	transferRevertsWhenPaused	Revert Condition	transfer reverts when paused
B-54	convertToSharesRoundTrip	Integrity	Round-trip convertToAssets(convertToShares(assets)) <= assets — shares conversion never inflates value
B-55	convertToAssetsRoundTrip	Integrity	Round-trip convertToShares(convertToAssets(shares)) <= shares — assets conversion never inflates value
B-56	depositIntegrity	Integrity	Deposit: receiver gains exact shares, caller loses exact assets, totalSupply increases
B-57	mintIntegrity	Integrity	Mint: receiver gains exact shares, caller loses exact assets, totalSupply increases

B-58	withdrawShareIntegrity	Integrity	Withdraw: owner loses exact shares, totalSupply decreases
B-59	redeemShareIntegrity	Integrity	Redeem: owner loses exact shares, totalSupply decreases
B-60	solvencyAfterDeposit	Integrity	totalSupply > 0 => asset balance > 0 preserved after deposit
B-61	solvencyAfterMint	Integrity	totalSupply > 0 => asset balance > 0 preserved after mint
B-64	maxWithdrawUpperBound	Revert Condition	withdraw reverts if assets exceed maxWithdraw(owner)
B-65	maxRedeemUpperBound	Revert Condition	redeem reverts if shares exceed maxRedeem(owner)
B-66	previewDepositNeverExceedsFreeConversion	Integrity	previewDeposit/assets) <= convertToShares/assets) (deposit fee is non-negative)
B-67	previewMintNeverBelowFreeConversion	Integrity	previewMint/shares) >= convertToAssets/shares) (mint fee is non-negative)
B-68	previewWithdrawNeverBelowFreeConversion	Integrity	previewWithdraw/assets) >= convertToShares/assets) (withdrawal fee is non-negative)

IBTCYHub (Subscription/Redemption Hub)

22 properties covering counter monotonicity, fee bounds, access control, request lifecycle, pause enforcement, minimum amount validation, and admin safety.

ID	Name	Type	Description
H-1	subscriptionCounterMonotonicity	Parametric	subscriptionRequestCounter only increases
H-2	redemptionCounterMonotonicity	Parametric	redemptionRequestCounter only increases
H-3	instantRedemptionFeeBounded	Invariant	instantRedemptionFee <= MAX_FEE_BPS
H-4	regularRedemptionFeeBounded	Invariant	regularRedemptionFee <= MAX_FEE_BPS
H-5	instantSubscriptionFeeBounded	Invariant	instantSubscriptionFee <= MAX_FEE_BPS
H-6	regularSubscriptionFeeBounded	Invariant	regularSubscriptionFee <= MAX_FEE_BPS
H-7	onlyConfigAdminCanSetFees	Access Control (Parametric)	Only CONFIG_ADMIN can change fee values (4 fee setters)
H-8	processSubscriptionsOnlyByManager	Access Control	processSubscriptions reverts if caller lacks MANAGER_ROLE
H-9	processRedemptionsOnlyByManager	Access Control	processRedemptions reverts if caller lacks MANAGER_ROLE
H-10	requestSubscriptionCreatesRecord	Integrity	requestSubscription creates a valid deposit record with matching user and amount
H-11	cancelSubscriptionAuth	Access Control	cancelSubscription reverts unless caller is depositor or has MANAGER_ROLE
H-12	cancelRedemptionAuth	Access Control	cancelRedemption reverts unless caller is redeemer or has MANAGER_ROLE
H-13	rescueRedemptionOnlyByAdmin	Access Control	rescueRedemption reverts if caller lacks DEFAULT_ADMIN_ROLE
H-14	requestSubscriptionSanity	Sanity	requestSubscription is reachable
H-15	processSubscriptionsRevertsWhenPaused	Revert Condition	processSubscriptions reverts when subscriptionPaused is true
H-16	setMinimumDepositAmountRevertsOnZero	Revert Condition	setMinimumDepositAmount(0) reverts
H-17	setMinimumRedemptionAmountRevertsOnZero	Revert Condition	setMinimumRedemptionAmount(0) reverts
H-18	cannotRenounceLastAdmin	Revert Condition	Cannot renounce the last DEFAULT_ADMIN_ROLE
H-19	cannotRevokeLastAdmin	Revert Condition	Cannot revoke the last DEFAULT_ADMIN_ROLE
H-20	processRedemptionsRevertsWhenPaused	Revert Condition	processRedemptions reverts when redemptionPaused is true

H-21	requestSubscriptionAuth	Access Control	requestSubscription reverts if caller is neither user nor has REQUEST_ROLE
H-22	requestRedemptionAuth	Access Control	requestRedemption reverts if caller is neither user nor has REQUEST_ROLE

Pricer (Price Oracle)

16 properties covering price ID monotonicity, price validation, access control, rate limiting, timestamp validation, and admin setter authorization.

ID	Name	Type	Description
P-1	currentPriceIdOnlyIncreases	Parametric	currentPriceId only increases (monotonicity)
P-2	addPriceRevertsOnZeroPrice	Revert Condition	addPrice reverts if price is zero
P-3	addPriceRateLimiting	Revert Condition	addPrice reverts if called before MIN_PRICE_UPDATE_INTERVAL elapses
P-4	updatePriceRevertsOnZeroPrice	Revert Condition	updatePrice reverts if new price is zero
P-5	updatePriceRevertsOnNonexistentId	Revert Condition	updatePrice reverts if price ID doesn't exist (stored price is zero)
P-6	onlyPriceUpdaterCanMutatePrices	Access Control (Parametric)	Only PRICE_UPDATE_ROLE can call addPrice, updatePrice, addCurrentPrice
P-7	onlyAdminCanSetPriceFeed	Access Control	Only DEFAULT_ADMIN_ROLE can call setPriceFeed
P-8	addPriceStoresCorrectly	Integrity	After addPrice, stored price and timestamp match the input, and price ID incremented by 1
P-9	addPriceSanity	Sanity	addPrice is reachable
P-10	updatePriceSanity	Sanity	updatePrice is reachable
P-11	cannotRenounceLastAdmin	Revert Condition	Cannot renounce the last DEFAULT_ADMIN_ROLE
P-12	cannotRevokeLastAdmin	Revert Condition	Cannot revoke the last DEFAULT_ADMIN_ROLE
P-13	onlyAdminCanCallPricerSetters	Access Control (Parametric)	Only DEFAULT_ADMIN_ROLE can call 5 admin setter functions
P-14	addPriceRevertsOnFutureTimestamp	Revert Condition	addPrice reverts if timestamp is in the future
P-15	setChainlinkPriceBoundsRevertsOnInvalidBounds	Revert Condition	setChainlinkPriceBounds reverts if minPrice >= maxPrice (both positive)
P-16	addPriceTimestampMonotonicity	Integrity	lastPriceUpdateTime never decreases after addPrice

DepositWithdraw (Treasury)

12 properties covering whitelist enforcement, deposit pause, authorization, treasury setter validation, and admin safety.

ID	Name	Type	Description
D-1	depositRevertsIfUserNotWhitelisted	Revert Condition	deposit reverts if caller is not user-whitelisted
D-2	depositRevertsIfTokenNotWhitelisted	Revert Condition	deposit reverts if token is not whitelisted
D-3	depositRevertsWhenPaused	Revert Condition	deposit reverts when deposits are paused
D-4	withdrawOnlyByTreasury	Access Control	withdraw reverts if caller is not the treasury address
D-5	withdrawTransferOnlyByManager	Access Control	withdrawTransfer reverts if caller lacks MANAGER_ROLE
D-6	onlyManagerCanManageWhitelist	Access Control (Parametric)	Only MANAGER_ROLE can manage whitelist entries (6 functions)

D-7	onlyAdminCanSetTreasury	Access Control	Only DEFAULT_ADMIN_ROLE can set treasury address
D-8	setTreasuryRevertsOnZeroAddress	Revert Condition	setTreasury reverts on zero address
D-9	depositSanity	Sanity	deposit is reachable
D-10	withdrawTransferSanity	Sanity	withdrawTransfer is reachable
D-11	cannotRenounceLastAdmin	Revert Condition	Cannot renounce the last DEFAULT_ADMIN_ROLE
D-12	cannotRevokeLastAdmin	Revert Condition	Cannot revoke the last DEFAULT_ADMIN_ROLE

Setup and Execution

The Certora Prover can be run either remotely (using Certora's cloud infrastructure) or locally (building from source). Both modes share the same initial setup steps.

Common Setup (Steps 1–4)

The instructions below are for Ubuntu 24.04. For step-by-step installation details refer to this setup [tutorial](#).

1. Install Java (tested with JDK 21)

```
sudo apt update
sudo apt install default-jre
java -version
```

2. Install [pipx](#) — installs Python CLI tools in isolated environments, avoiding dependency conflicts

```
sudo apt install pipx
pipx ensurepath
```

3. Install Certora CLI. To match a specific prover version, pin it explicitly (e.g. `certora-cli==8.8.0`)

```
pipx install certora-cli
```

4. Install solc-select and the Solidity compiler version required by the project

```
pipx install solc-select
solc-select install 0.8.30
solc-select use 0.8.30
```

Remote Execution

5. Set up Certora key. You can get a free key through the Certora [Discord](#) or on their website. Once you have it, export it:

```
echo "export CERTORAKEY=<your_certora_api_key>" >> ~/.bashrc
source ~/.bashrc
```

Note: If a local prover is installed (see below), it takes priority. To force remote execution, add the `--server production` flag:

```
certoraRun certora/conf/IBTCY.conf --server production
```

Local Execution

Follow the full build instructions in the [CertoraProver repository \(v8.8.0\)](#). Once the local prover is installed it takes priority over the remote cloud by default. Tested on Ubuntu 24.04.

1. Install prerequisites

```
# JDK 19+
sudo apt install openjdk-21-jdk

# SMT solvers (z3 and cvc5 are required, others are optional)
# Download binaries and place them in PATH:
# z3: https://github.com/Z3Prover/z3/releases
# cvc5: https://github.com/cvc5/cvc5/releases

# LLVM tools
sudo apt install llvm
```

```
# Rust 1.81.0+
curl --proto 'https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
cargo install rustfilt

# Graphviz (optional, for visual reports)
sudo apt install graphviz
```

2. Set up build output directory

```
export CERTORA="$HOME/CertoraProver/target/installed/"
mkdir -p "$CERTORA"
export PATH="$CERTORA:$PATH"
```

3. Clone and build

```
git clone --recurse-submodules https://github.com/Certora/CertoraProver.git
cd CertoraProver
git checkout tags/8.8.0
./gradlew assemble
```

4. Verify installation with test example

```
certoraRun.py -h
cd Public/TestEVM/Counter
certoraRun counter.conf
```

Running Verification

Each contract has its own configuration file. Run all rules for a contract:

```
# IBTCY – token conservation, access control, force transfer safety
certoraRun certora/conf/IBTCY.conf

# BTCY – fee bounds, deny list, allowlist, MIN_SHARES, admin safety
certoraRun certora/conf/BTCY.conf

# BTCY EIP-4626 – preview compliance, round-trip, integrity, solvency, fee
certoraRun certora/conf/BTCY_EIP4626.conf

# IBTCYHub – counter monotonicity, fee bounds, access control, request lifecycle
certoraRun certora/conf/IBTCYHub.conf

# Pricer – price validation, access control, rate limiting
certoraRun certora/conf/Pricer.conf

# DepositWithdraw – whitelist enforcement, authorization, treasury safety
certoraRun certora/conf/DepositWithdraw.conf
```

To run a specific rule (useful if a full conf times out):

```
certoraRun certora/conf/IBTCY.conf --rule onlyMintIncreasesTotalSupply
certoraRun certora/conf/BTCY.conf --rule withdrawalFeeBounded
certoraRun certora/conf/BTCY_EIP4626.conf --rule previewDepositMatchesDeposit
certoraRun certora/conf/IBTCYHub.conf --rule subscriptionCounterMonotonicity
```

To check compilation without submitting to the prover:

```
certoraRun certora/conf/IBTCY.conf --compilation_steps_only
```

Resources

To learn more about Certora formal verification:

- [Updraft Assembly & Formal Verification Course](#) — Comprehensive video course covering assembly and formal verification from the ground up
- [Find Highs Using Certora Formal Verification](#) — Practical guide with a companion [repo](#) containing simplified examples based on real code and bugs from private audits
- [RareSkills Certora Book](#) — Structured tutorial covering CVL syntax, patterns, and common pitfalls
- [Alex FV Resources](#) — Curated collection of formal verification resources, examples, and references
- [Certora Tutorials](#) — Official Certora documentation and guided tutorials